Maybe the trickiest question is how to maintain women's preferences to keep step (4) efficient. Consider a step of the algorithm, when man m proposes to a woman w. Assume w is already engaged, and her current partner is $m' =$`Current`$[w]$. We would like to decide in $O(1)$ time if woman w prefers m or m'. Keeping the women's preferences in an array `WomanPref`, analogous to the one we used for men, does not work, as we would need to walk through w's list one by one, taking $O(n)$ time to find m and m' on the list. While $O(n)$ is still polynomial, we can do a lot better if we build an auxiliary data structure at the beginning.

At the start of the algorithm, we create an $n \times n$ array `Ranking`, where `Ranking`$[w, m]$ contains the rank of man m in the sorted order of w's preferences. By a single pass through w's preference list, we can create this array in linear time for each woman, for a total initial time investment proportional to n^2. Then, to decide which of m or m' is preferred by w, we simply compare the values `Ranking`$[w, m]$ and `Ranking`$[w, m']$.

This allows us to execute step (4) in constant time, and hence we have everything we need to obtain the desired running time.

(2.10) *The data structures described above allow us to implement the G-S algorithm in $O(n^2)$ time.*

2.4 A Survey of Common Running Times

When trying to analyze a new algorithm, it helps to have a rough sense of the "landscape" of different running times. Indeed, there are styles of analysis that recur frequently, and so when one sees running-time bounds like $O(n)$, $O(n \log n)$, and $O(n^2)$ appearing over and over, it's often for one of a very small number of distinct reasons. Learning to recognize these common styles of analysis is a long-term goal. To get things under way, we offer the following survey of common running-time bounds and some of the typical approaches that lead to them.

Earlier we discussed the notion that most problems have a natural "search space"—the set of all possible solutions—and we noted that a unifying theme in algorithm design is the search for algorithms whose performance is more efficient than a brute-force enumeration of this search space. In approaching a new problem, then, it often helps to think about two kinds of bounds: one on the running time you hope to achieve, and the other on the size of the problem's natural search space (and hence on the running time of a brute-force algorithm for the problem). The discussion of running times in this section will begin in many cases with an analysis of the brute-force algorithm, since it is a useful

way to get one's bearings with respect to a problem; the task of improving on such algorithms will be our goal in most of the book.

Linear Time

An algorithm that runs in $O(n)$, or linear, time has a very natural property: its running time is at most a constant factor times the size of the input. One basic way to get an algorithm with this running time is to process the input in a single pass, spending a constant amount of time on each item of input encountered. Other algorithms achieve a linear time bound for more subtle reasons. To illustrate some of the ideas here, we consider two simple linear-time algorithms as examples.

Computing the Maximum Computing the maximum of n numbers, for example, can be performed in the basic "one-pass" style. Suppose the numbers are provided as input in either a list or an array. We process the numbers $a_1, a_2, \ldots, a_n$ in order, keeping a running estimate of the maximum as we go. Each time we encounter a number a_i, we check whether a_i is larger than our current estimate, and if so we update the estimate to a_i.

```
max = a_1
For i = 2 to n
  If a_i > max then
    set max = a_i
  Endif
Endfor
```

In this way, we do constant work per element, for a total running time of $O(n)$.

Sometimes the constraints of an application force this kind of one-pass algorithm on you—for example, an algorithm running on a high-speed switch on the Internet may see a stream of packets flying past it, and it can try computing anything it wants to as this stream passes by, but it can only perform a constant amount of computational work on each packet, and it can't save the stream so as to make subsequent scans through it. Two different subareas of algorithms, *online algorithms* and *data stream algorithms*, have developed to study this model of computation.

Merging Two Sorted Lists Often, an algorithm has a running time of $O(n)$, but the reason is more complex. We now describe an algorithm for merging two sorted lists that stretches the one-pass style of design just a little, but still has a linear running time.

Suppose we are given two lists of n numbers each, $a_1, a_2, \ldots, a_n$ and $b_1, b_2, \ldots, b_n$, and each is already arranged in ascending order. We'd like to

merge these into a single list $c_1, c_2, \ldots, c_{2n}$ that is also arranged in ascending order. For example, merging the lists 2, 3, 11, 19 and 4, 9, 16, 25 results in the output 2, 3, 4, 9, 11, 16, 19, 25.

To do this, we could just throw the two lists together, ignore the fact that they're separately arranged in ascending order, and run a sorting algorithm. But this clearly seems wasteful; we'd like to make use of the existing order in the input. One way to think about designing a better algorithm is to imagine performing the merging of the two lists by hand: suppose you're given two piles of numbered cards, each arranged in ascending order, and you'd like to produce a single ordered pile containing all the cards. If you look at the top card on each stack, you know that the smaller of these two should go first on the output pile; so you could remove this card, place it on the output, and now iterate on what's left.

In other words, we have the following algorithm.

```
To merge sorted lists A = a1, ..., an and B = b1, ..., bn:
  Maintain a Current pointer into each list, initialized to
    point to the front elements
  While both lists are nonempty:
    Let ai and bj be the elements pointed to by the Current pointer
    Append the smaller of these two to the output list
    Advance the Current pointer in the list from which the
      smaller element was selected
  EndWhile
  Once one list is empty, append the remainder of the other list
      to the output
```

See Figure 2.2 for a picture of this process.

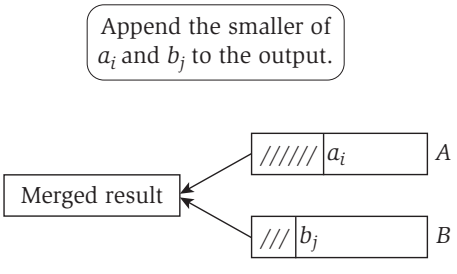

Figure 2.2 To merge sorted lists A and B, we repeatedly extract the smaller item from the front of the two lists and append it to the output.

Now, to show a linear-time bound, one is tempted to describe an argument like what worked for the maximum-finding algorithm: "We do constant work per element, for a total running time of $O(n)$." But it is actually not true that we do only constant work per element. Suppose that n is an even number, and consider the lists $A = 1, 3, 5, \ldots, 2n-1$ and $B = n, n+2, n+4, \ldots, 3n-2$. The number b_1 at the front of list B will sit at the front of the list for $n/2$ iterations while elements from A are repeatedly being selected, and hence it will be involved in $\Omega(n)$ comparisons. Now, it is true that each element can be involved in at most $O(n)$ comparisons (at worst, it is compared with each element in the other list), and if we sum this over all elements we get a running-time bound of $O(n^2)$. This is a correct bound, but we can show something much stronger.

The better way to argue is to bound the number of iterations of the `While` loop by an "accounting" scheme. Suppose we *charge* the cost of each iteration to the element that is selected and added to the output list. An element can be charged only once, since at the moment it is first charged, it is added to the output and never seen again by the algorithm. But there are only $2n$ elements total, and the cost of each iteration is accounted for by a charge to some element, so there can be at most $2n$ iterations. Each iteration involves a constant amount of work, so the total running time is $O(n)$, as desired.

While this merging algorithm iterated through its input lists in order, the "interleaved" way in which it processed the lists necessitated a slightly subtle running-time analysis. In Chapter 3 we will see linear-time algorithms for graphs that have an even more complex flow of control: they spend a constant amount of time on each node and edge in the underlying graph, but the order in which they process the nodes and edges depends on the structure of the graph.

$O(n \log n)$ Time

$O(n \log n)$ is also a very common running time, and in Chapter 5 we will see one of the main reasons for its prevalence: it is the running time of any algorithm that splits its input into two equal-sized pieces, solves each piece recursively, and then combines the two solutions in linear time.

Sorting is perhaps the most well-known example of a problem that can be solved this way. Specifically, the *Mergesort* algorithm divides the set of input numbers into two equal-sized pieces, sorts each half recursively, and then merges the two sorted halves into a single sorted output list. We have just seen that the merging can be done in linear time; and Chapter 5 will discuss how to analyze the recursion so as to get a bound of $O(n \log n)$ on the overall running time.

One also frequently encounters $O(n \log n)$ as a running time simply because there are many algorithms whose most expensive step is to sort the input. For example, suppose we are given a set of n time-stamps $x_1, x_2, \ldots, x_n$ on which copies of a file arrived at a server, and we'd like to find the largest interval of time between the first and last of these time-stamps during which no copy of the file arrived. A simple solution to this problem is to first sort the time-stamps $x_1, x_2, \ldots, x_n$ and then process them in sorted order, determining the sizes of the gaps between each number and its successor in ascending order. The largest of these gaps is the desired subinterval. Note that this algorithm requires $O(n \log n)$ time to sort the numbers, and then it spends constant work on each number in ascending order. In other words, the remainder of the algorithm after sorting follows the basic recipe for linear time that we discussed earlier.

Quadratic Time

Here's a basic problem: suppose you are given n points in the plane, each specified by (x, y) coordinates, and you'd like to find the pair of points that are closest together. The natural brute-force algorithm for this problem would enumerate all pairs of points, compute the distance between each pair, and then choose the pair for which this distance is smallest.

What is the running time of this algorithm? The number of pairs of points is $\binom{n}{2} = \frac{n(n-1)}{2}$, and since this quantity is bounded by $\frac{1}{2}n^2$, it is $O(n^2)$. More crudely, the number of pairs is $O(n^2)$ because we multiply the number of ways of choosing the first member of the pair (at most n) by the number of ways of choosing the second member of the pair (also at most n). The distance between points (x_i, y_i) and (x_j, y_j) can be computed by the formula $\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$ in constant time, so the overall running time is $O(n^2)$. This example illustrates a very common way in which a running time of $O(n^2)$ arises: performing a search over all pairs of input items and spending constant time per pair.

Quadratic time also arises naturally from a pair of *nested loops*: An algorithm consists of a loop with $O(n)$ iterations, and each iteration of the loop launches an internal loop that takes $O(n)$ time. Multiplying these two factors of n together gives the running time.

The brute-force algorithm for finding the closest pair of points can be written in an equivalent way with two nested loops:

```
For each input point (x_i, y_i)
  For each other input point (x_j, y_j)
    Compute distance d = sqrt((x_i - x_j)^2 + (y_i - y_j)^2)
```

```
    If d is less than the current minimum, update minimum to d
  Endfor
Endfor
```

Note how the "inner" loop, over (x_j, y_j), has $O(n)$ iterations, each taking constant time; and the "outer" loop, over (x_i, y_i), has $O(n)$ iterations, each invoking the inner loop once.

It's important to notice that the algorithm we've been discussing for the Closest-Pair Problem really is just the brute-force approach: the natural search space for this problem has size $O(n^2)$, and we're simply enumerating it. At first, one feels there is a certain inevitability about this quadratic algorithm—we have to measure all the distances, don't we?—but in fact this is an illusion. In Chapter 5 we describe a very clever algorithm that finds the closest pair of points in the plane in only $O(n \log n)$ time, and in Chapter 13 we show how randomization can be used to reduce the running time to $O(n)$.

Cubic Time

More elaborate sets of nested loops often lead to algorithms that run in $O(n^3)$ time. Consider, for example, the following problem. We are given sets $S_1, S_2, \ldots, S_n$, each of which is a subset of $\{1, 2, \ldots, n\}$, and we would like to know whether some pair of these sets is disjoint—in other words, has no elements in common.

What is the running time needed to solve this problem? Let's suppose that each set S_i is represented in such a way that the elements of S_i can be listed in constant time per element, and we can also check in constant time whether a given number p belongs to S_i. The following is a direct way to approach the problem.

```
For pair of sets S_i and S_j
  Determine whether S_i and S_j have an element in common
Endfor
```

This is a concrete algorithm, but to reason about its running time it helps to open it up (at least conceptually) into three nested loops.

```
For each set S_i
  For each other set S_j
    For each element p of S_i
      Determine whether p also belongs to S_j
    Endfor
    If no element of S_i belongs to S_j then
```

```
      Report that S_i and S_j are disjoint
    Endif
  Endfor
Endfor
```

Each of the sets has maximum size $O(n)$, so the innermost loop takes time $O(n)$. Looping over the sets S_j involves $O(n)$ iterations around this innermost loop; and looping over the sets S_i involves $O(n)$ iterations around this. Multiplying these three factors of n together, we get the running time of $O(n^3)$.

For this problem, there are algorithms that improve on $O(n^3)$ running time, but they are quite complicated. Furthermore, it is not clear whether the improved algorithms for this problem are practical on inputs of reasonable size.

$O(n^k)$ Time

In the same way that we obtained a running time of $O(n^2)$ by performing brute-force search over all pairs formed from a set of n items, we obtain a running time of $O(n^k)$ for any constant k when we search over all subsets of size k.

Consider, for example, the problem of finding independent sets in a graph, which we discussed in Chapter 1. Recall that a set of nodes is independent if no two are joined by an edge. Suppose, in particular, that for some fixed constant k, we would like to know if a given n-node input graph G has an independent set of size k. The natural brute-force algorithm for this problem would enumerate all subsets of k nodes, and for each subset S it would check whether there is an edge joining any two members of S. That is,

```
For each subset S of k nodes
  Check whether S constitutes an independent set
  If S is an independent set then
    Stop and declare success
  Endif
Endfor
If no k-node independent set was found then
  Declare failure
Endif
```

To understand the running time of this algorithm, we need to consider two quantities. First, the total number of k-element subsets in an n-element set is

$$\binom{n}{k} = \frac{n(n-1)(n-2)\cdots(n-k+1)}{k(k-1)(k-2)\cdots(2)(1)} \leq \frac{n^k}{k!}.$$

Since we are treating k as a constant, this quantity is $O(n^k)$. Thus, the outer loop in the algorithm above will run for $O(n^k)$ iterations as it tries all k-node subsets of the n nodes of the graph.

Inside this loop, we need to test whether a given set S of k nodes constitutes an independent set. The definition of an independent set tells us that we need to check, for each pair of nodes, whether there is an edge joining them. Hence this is a search over pairs, like we saw earlier in the discussion of quadratic time; it requires looking at $\binom{k}{2}$, that is, $O(k^2)$, pairs and spending constant time on each.

Thus the total running time is $O(k^2 n^k)$. Since we are treating k as a constant here, and since constants can be dropped in $O(\cdot)$ notation, we can write this running time as $O(n^k)$.

Independent Set is a principal example of a problem believed to be computationally hard, and in particular it is believed that no algorithm to find k-node independent sets in arbitrary graphs can avoid having some dependence on k in the exponent. However, as we will discuss in Chapter 10 in the context of a related problem, even once we've conceded that brute-force search over k-element subsets is necessary, there can be different ways of going about this that lead to significant differences in the efficiency of the computation.

Beyond Polynomial Time

The previous example of the Independent Set Problem starts us rapidly down the path toward running times that grow faster than any polynomial. In particular, two kinds of bounds that come up very frequently are 2^n and $n!$, and we now discuss why this is so.

Suppose, for example, that we are given a graph and want to find an independent set of *maximum* size (rather than testing for the existence of one with a given number of nodes). Again, people don't know of algorithms that improve significantly on brute-force search, which in this case would look as follows.

```
For each subset S of nodes
  Check whether S constitutes an independent set
  If S is a larger independent set than the largest seen so far then
    Record the size of S as the current maximum
  Endif
Endfor
```

This is very much like the brute-force algorithm for k-node independent sets, except that now we are iterating over *all* subsets of the graph. The total number

of subsets of an n-element set is 2^n, and so the outer loop in this algorithm will run for 2^n iterations as it tries all these subsets. Inside the loop, we are checking all pairs from a set S that can be as large as n nodes, so each iteration of the loop takes at most $O(n^2)$ time. Multiplying these two together, we get a running time of $O(n^2 2^n)$.

Thus see that 2^n arises naturally as a running time for a search algorithm that must consider all subsets. In the case of Independent Set, something at least nearly this inefficient appears to be necessary; but it's important to keep in mind that 2^n is the size of the search space for many problems, and for many of them we will be able to find highly efficient polynomial-time algorithms. For example, a brute-force search algorithm for the Interval Scheduling Problem that we saw in Chapter 1 would look very similar to the algorithm above: try all subsets of intervals, and find the largest subset that has no overlaps. But in the case of the Interval Scheduling Problem, as opposed to the Independent Set Problem, we will see (in Chapter 4) how to find an optimal solution in $O(n \log n)$ time. This is a recurring kind of dichotomy in the study of algorithms: two algorithms can have very similar-looking search spaces, but in one case you're able to bypass the brute-force search algorithm, and in the other you aren't.

The function $n!$ grows even more rapidly than 2^n, so it's even more menacing as a bound on the performance of an algorithm. Search spaces of size $n!$ tend to arise for one of two reasons. First, $n!$ is the number of ways to match up n items with n other items—for example, it is the number of possible perfect matchings of n men with n women in an instance of the Stable Matching Problem. To see this, note that there are n choices for how we can match up the first man; having eliminated this option, there are $n-1$ choices for how we can match up the second man; having eliminated these two options, there are $n-2$ choices for how we can match up the third man; and so forth. Multiplying all these choices out, we get $n(n-1)(n-2)\cdots(2)(1) = n!$

Despite this enormous set of possible solutions, we were able to solve the Stable Matching Problem in $O(n^2)$ iterations of the proposal algorithm. In Chapter 7, we will see a similar phenomenon for the Bipartite Matching Problem we discussed earlier; if there are n nodes on each side of the given bipartite graph, there can be up to $n!$ ways of pairing them up. However, by a fairly subtle search algorithm, we will be able to find the largest bipartite matching in $O(n^3)$ time.

The function $n!$ also arises in problems where the search space consists of all ways to arrange n items in order. A basic problem in this genre is the Traveling Salesman Problem: given a set of n cities, with distances between all pairs, what is the shortest tour that visits all cities? We assume that the salesman starts and ends at the first city, so the crux of the problem is the

implicit search over all orders of the remaining $n - 1$ cities, leading to a search space of size $(n - 1)!$. In Chapter 8, we will see that Traveling Salesman is another problem that, like Independent Set, belongs to the class of NP-complete problems and is believed to have no efficient solution.

Sublinear Time

Finally, there are cases where one encounters running times that are asymptotically smaller than linear. Since it takes linear time just to read the input, these situations tend to arise in a model of computation where the input can be "queried" indirectly rather than read completely, and the goal is to minimize the amount of querying that must be done.

Perhaps the best-known example of this is the binary search algorithm. Given a sorted array A of n numbers, we'd like to determine whether a given number p belongs to the array. We could do this by reading the entire array, but we'd like to do it much more efficiently, taking advantage of the fact that the array is sorted, by carefully *probing* particular entries. In particular, we probe the middle entry of A and get its value—say it is q—and we compare q to p. If $q = p$, we're done. If $q > p$, then in order for p to belong to the array A, it must lie in the lower half of A; so we ignore the upper half of A from now on and recursively apply this search in the lower half. Finally, if $q < p$, then we apply the analogous reasoning and recursively search in the upper half of A.

The point is that in each step, there's a region of A where p might possibly be; and we're shrinking the size of this region by a factor of two with every probe. So how large is the "active" region of A after k probes? It starts at size n, so after k probes it has size at most $(\frac{1}{2})^k n$.

Given this, how long will it take for the size of the active region to be reduced to a constant? We need k to be large enough so that $(\frac{1}{2})^k = O(1/n)$, and to do this we can choose $k = \log_2 n$. Thus, when $k = \log_2 n$, the size of the active region has been reduced to a constant, at which point the recursion bottoms out and we can search the remainder of the array directly in constant time.

So the running time of binary search is $O(\log n)$, because of this successive shrinking of the search region. In general, $O(\log n)$ arises as a time bound whenever we're dealing with an algorithm that does a constant amount of work in order to throw away a constant *fraction* of the input. The crucial fact is that $O(\log n)$ such iterations suffice to shrink the input down to constant size, at which point the problem can generally be solved directly.

2.5 A More Complex Data Structure: Priority Queues

Our primary goal in this book was expressed at the outset of the chapter: we seek algorithms that improve qualitatively on brute-force search, and in general we use polynomial-time solvability as the concrete formulation of this. Typically, achieving a polynomial-time solution to a nontrivial problem is not something that depends on fine-grained implementation details; rather, the difference between exponential and polynomial is based on overcoming higher-level obstacles. Once one has an efficient algorithm to solve a problem, however, it is often possible to achieve further improvements in running time by being careful with the implementation details, and sometimes by using more complex data structures.

Some complex data structures are essentially tailored for use in a single kind of algorithm, while others are more generally applicable. In this section, we describe one of the most broadly useful sophisticated data structures, the *priority queue.* Priority queues will be useful when we describe how to implement some of the graph algorithms developed later in the book. For our purposes here, it is a useful illustration of the analysis of a data structure that, unlike lists and arrays, must perform some nontrivial processing each time it is invoked.

The Problem

In the implementation of the Stable Matching algorithm in Section 2.3, we discussed the need to maintain a dynamically changing set S (such as the set of all free men in that case). In such situations, we want to be able to add elements to and delete elements from the set S, and we want to be able to select an element from S when the algorithm calls for it. A priority queue is designed for applications in which elements have a *priority value,* or *key,* and each time we need to select an element from S, we want to take the one with highest priority.

A priority queue is a data structure that maintains a set of elements S, where each element $v \in S$ has an associated value $\texttt{key}(v)$ that denotes the priority of element v; smaller keys represent higher priorities. Priority queues support the addition and deletion of elements from the set, and also the selection of the element with smallest key. Our implementation of priority queues will also support some additional operations that we summarize at the end of the section.

A motivating application for priority queues, and one that is useful to keep in mind when considering their general function, is the problem of managing